Lahore Ride-Hailing

Technical Assessment Response

Assessment: Lead Full Stack / Backend Engineer

Review date: 2 November 2026

Project overview

This project implements the ride-booking workflow described in the assessment. Riders choose a pickup, destination, and ride category, review a fare estimate and pickup ETA, and request a ride. They can follow the ride's status and cancel when permitted. Drivers can change availability, accept eligible requests, and complete assigned trips.

The frontend uses Next.js and TypeScript. The backend is a Go API, with Supabase Auth for authentication and PostgreSQL for application data. The web and API services run separately using the README instructions.

Assessment requirements

- **Rider interface:** Pickup and destination inputs, Economy and Comfort categories, fare and ETA estimates, booking confirmation, ride status, and cancellation. The interface includes loading, success, empty, and error states.
- **Driver workflow:** Saved availability, eligible requests, ride acceptance, an active assignment view, and trip completion.
- **API:** Create, retrieve, accept, cancel, and complete operations, with supporting endpoints for quotes, locations, availability, and current assignments.
- **Database:** Versioned SQL migrations for users, driver profiles, locations, rides, assignments, pricing, and idempotency records. Core state is stored in PostgreSQL.
- **Reliability:** Transactions, row locks, constraints, and idempotency keys protect assignment and booking retries.
- **Maps:** Google Maps displays pickup and destination points on booking, rider status, and active driver trip views. Riders can select points within the configured Lahore bounds and edit their labels. The connecting line is approximate. The assessment permits a simplified map.
- **Submission material:** Setup instructions, SQL migrations, automated tests, OpenAPI documentation, architecture diagrams, design notes, and known limitations are included in the repository.

Architecture questions

1. Why did you choose this technology stack and architecture?

I chose Next.js and TypeScript for shared interface components and typed API data. Next.js forwards browser API requests to Go through the application origin. Supabase Auth manages sign-in and session refresh.

Go provides an explicit backend for validation, pricing, matching, and ride transitions. Chi handles HTTP routing, while pgx provides direct access to PostgreSQL transactions and row locks. PostgreSQL stores the authoritative state and enforces important rules through constraints.

The backend is a modular monolith: HTTP handlers, business rules, and persistence are separate parts of one service. This keeps the project easy to run and review. The main operational cost is maintaining both a Node process and a Go process.

Requests flow from the browser through Next.js to the Go API and PostgreSQL. Supabase Auth issues access tokens; Go validates them and reads the user's application role from the database. Architecture and ride-state diagrams are included in the architecture notes.

2. How do you prevent two drivers from accepting the same ride?

Acceptance runs inside a PostgreSQL transaction. The API locks the ride row, then the acting driver's row, in a consistent order. It checks that the ride is still open and that the driver is available and eligible for its city, category, and pickup radius.

The transaction creates the assignment, marks the ride as accepted, and marks the driver as busy before committing. A unique assignment per ride prevents a second assignment. A partial unique index prevents a driver from holding more than one active assignment.

If two drivers accept simultaneously, one transaction completes first. The other then sees the updated state and receives a conflict response. This protection works across multiple API instances because PostgreSQL coordinates the locks and constraints.

3. How do you handle retries and duplicate requests?

The browser creates an idempotency key for each booking attempt and retains it if the response is lost or times out. The API stores that key, a fingerprint of the booking input, and the resulting ride in one transaction.

Repeating the same input with the same key returns the original ride. Reusing the key with different input returns a conflict. A separate database constraint prevents a rider from creating a second open ride with another key.

The fingerprint includes the normalized pickup, destination, and category. For custom map points, it also includes their labels and coordinates. Those points are saved in the ride transaction.

4. How does the frontend respond when an API request fails?

The interface shows a pending state and disables repeated actions while a request is running. Validation and API failures appear as readable messages. The API client checks error-response fields and uses fallback text for malformed data, including object-shaped login errors.

An unauthorized response clears the browser authentication session. A conflict refreshes the relevant view from the server. If ride creation has an uncertain result, the browser checks for an existing current ride and retains the original idempotency key for a safe retry.

Active ride status is refreshed by polling. Polling stops when the ride is completed or cancelled.

5. How would you scale to multiple cities and thousands of concurrent users?

I would first measure request latency, database pool use, query performance, and lock waits under representative load. The Go API can run as multiple stateless instances behind a load balancer. A managed connection pool would protect PostgreSQL from excessive connections.

City-specific availability, pricing, and matching rules would remain explicit in the data model and queries. For nearby-driver discovery, I would use PostGIS indexes and distance queries over recent driver coordinates. Location updates would need authentication and an expiry time so stale positions are excluded.

As traffic grows, I would replace frequent status polling with an appropriate update channel. Capacity for thousands of concurrent users has not been established by a load test and remains a validation step before production.

6. When would you introduce Redis, a message broker, or separate services?

I would introduce Redis when driver presence and nearby-driver lookups need faster updates or more reads than PostgreSQL should handle directly. Presence records would expire and be reconciled with durable application state.

A message broker becomes useful when notifications, analytics, or other consumers need reliable ride events and independent processing. A transactional outbox would save the ride change and its event together. Consumers would handle repeated deliveries safely.

I would separate services when a component needs independent scaling, ownership, or failure isolation. The current application does not yet justify the additional deployment and communication overhead.

7. How would you deploy, monitor, and roll back a faulty release?

I would deploy the web application and Go API as versioned artifacts, with managed PostgreSQL, HTTPS, and secrets supplied by the hosting platform. CI would run application tests, PostgreSQL integration checks, and migration-compatibility checks before release.

Database changes would use additive migrations first, followed by compatible application changes. Obsolete schema would be removed in a later release. Backups and restore procedures would be tested before accepting production traffic.

The API already produces structured logs and request IDs. I would add alerts for error rates, latency, database connections, lock waits, and failed ride transitions. A faulty application release could be rolled back to a compatible artifact. A migration that changed durable data would normally require a forward repair.

This is the proposed operating approach. Production hosting, monitoring, and restore procedures have not been demonstrated.

8. What are the three most important improvements before production?

1. **Dispatch and location accuracy.** Add authenticated driver location updates with expiry, dispatch offers with time limits, road-based routing, and pricing reviewed for actual operating costs.
2. **Identity and abuse protection.** Add controlled onboarding, account recovery, request rate limits, token-revocation procedures, and a reviewed policy for access to location and account data.
3. **Operational readiness.** Establish managed hosting, service metrics and alerts, tested backup restoration, safe migration procedures, and representative load tests.

Fare calculation

The server estimates distance using the Haversine formula and a configurable city-road factor, currently 1.3. The fare is the base fare plus estimated fuel cost plus distance operating cost.

Fuel cost uses the estimated distance, configured petrol price, and vehicle category's fuel efficiency. The API returns the rate's effective date and fare breakdown. Each booked ride stores a fixed snapshot of these values, so a later rate change does not alter its saved estimate.

Petrol prices are maintained in the pricing_rates database table. Updates are manual. Distance, fare, and pickup ETA remain estimates and do not account for live traffic.

Verification

The following checks were recorded on 2 November 2026:

- Go unit and HTTP tests, plus go vet, passed.
- The PostgreSQL integration suite passed against the configured Supabase database. It covered lifecycle rules, concurrent acceptance, duplicate requests, custom map points, and saved fare calculations.
- The latest frontend run passed all 24 tests across five files. Lint and TypeScript checks also passed.
- GitHub CI passed the API checks and the web tests, lint, TypeScript checks, and production build for the merged route-map change.
- Earlier browser tests passed with real Supabase Auth and Google Maps. They covered booking, driver availability, ride acceptance, completion reflected on the rider page, custom map pins, and fare breakdowns. These browser tests were not rerun for this document revision.

Database integration fixtures clean up their test rows. Browser journeys use demo accounts and leave completed or cancelled ride records. These checks do not establish production capacity under sustained load.

Known limitations

The application does not include address search, automatic address labels, road routing, live driver GPS, dispatch expiry or reassignment, payments, refunds, customer notifications, public signup, OAuth, or push updates. Driver positions are demo data. Fuel prices require manual updates. Production hosting, load testing, and backup restoration remain future work.

AI assistance

AI coding tools assisted with implementation and documentation. Their output was reviewed against the code and checked through automated tests, static checks, a production build, and the browser and database checks described above.

Supporting documents

- Setup and execution instructions
- API specification and API contract
- Architecture diagrams and design notes
- Known limitations and proposed improvements
- Database migrations